QDR·AI -- Application Documentation

QDR·AI is a Streamlit + FastAPI application that automates quality-assurance review of supplier hardness-inspection PDF reports (e.g. HRDR submittals) against project specifications. A 6-node LangGraph pipeline extracts table data from the submittal PDF, resolves each row's material grade, checks every hardness reading against the correct spec limit, and produces a scored, annotated compliance report -- using deterministic rules first and an LLM only for genuinely ambiguous/borderline cases.

---

1. Tech Stack

Layer	Tool / Library	Purpose
Frontend	**Streamlit** (app.py)	Single-page multi-tab UI, port 8501
Backend API	**FastAPI** + **Uvicorn** (src/api/main.py)	REST API, port 8000
Orchestration	**LangGraph** (src/agents/orchestrator.py)	6-node agent graph, linear topology
LLM runtime	**Ollama** (local, CPU-only)	qwen2.5-coder:7b (text), qwen2.5vl:7b (vision)
PDF parsing	**PyMuPDF (fitz)**, **pdfplumber**, **unstructured**	Text/table extraction, page rendering
OCR	**pytesseract** (Tesseract OCR) + **Pillow/OpenCV**	Scanned-page text + word-position extraction
Embeddings	**BAAI/bge-m3** via langchain-huggingface (sentence-transformers)	1024-dim embeddings for RAG
Vector store	**ChromaDB** via langchain-chroma	Parent/child chunk retrieval, persisted to data/chromadb/
Agent framework	**LangChain** (@tool functions in src/agents/tools.py)	Classification, retrieval, scoring tools
Config	python-dotenv (.env -> src/config.py)	Central environment configuration

Full pinned list: requirements.txt (no pyproject.toml -- pip/requirements-based).

---

2. High-Level Architecture

Both the Streamlit app and the FastAPI `/inspect` endpoint call the **same** `run_inspection_pipeline()` entrypoint -- the API is not required for the UI to work (`app.py` imports and calls the orchestrator directly), but exists for programmatic / external access to the same pipeline.

---

3. The 6-Node LangGraph Pipeline

Defined in src/agents/orchestrator.py. The graph is **strictly linear** -- every node has exactly one outgoing edge, no conditional routing:

- **entry**
- **[1] classify_and_retrieve** -- inline node, not a separate module
- **[2] ocr_extract** -- src/agents/ocr_extractor.py
- **[3] resolve_materials** -- src/agents/material_resolver_agent.py (Agent 2)
- **[4] audit_compliance** -- src/agents/compliance_auditor_agent.py (Agent 3)
- **[5] build_annotations** -- src/agents/annotation_builder.py (Agent 4)
- **[6] synthesize_report** -- src/agents/report_synthesizer_agent.py (Agent 5)
- **END**

The graph is compiled once (`_compiled_graph` module-level singleton) and reused across requests. `AGENT_MODE` (env var, default `"langgraph"`) selects this pipeline; any other value routes to `_run_legacy_pipeline`, a plain rule-based fallback (src/agents/qa_inspector_agent_legacy.py) also used automatically if the LangGraph pipeline raises an exception.

Stage 1 -- `classify_and_retrieve` (Orchestrator "Agent 1")

Defined inline in `orchestrator.py`, not a separate agent module -- it sequentially calls three `@tool` functions from src/agents/tools.py:

1. **`classify_document(submittal_text)`** -- pure regex, no LLM. Matches test-method patterns (Rockwell HRC / Brinell HBW / Vickers HV / UCI / Telebrinell) and material-grade patterns (P91/P92/P22/P11 aliases) against the first 3,000 characters. Defaults to `"Brinell HBW"` / `"Alloy 91 (P91)"` if nothing matches.
2. **`extract_requirements(guide_code, checked_criteria)`** -- if the user's uploaded Review Guide already produced checked checklist items (via `extract_checklist_items` in the UI), those are used directly. Otherwise falls back to a ChromaDB search (`"hardness test acceptance criteria shall mandatory"`) plus a hardcoded 4-item default checklist citing specs 700-006/700-022.
3. **`search_specs(query, guide_code, k=5)`** -- the RAG retrieval step. Builds a focused query (`"hardness testing acceptance criteria limits allowable maximum minimum " + query`) and searches the ChromaDB collection for `guide_code`; if fewer than 2 hits, retries with the unfocused query.

Each call is wrapped in a `StageTracker` for latency logging. Returns updates to `doc_type`, `test_method`, `material_grade`, `requirements`, `retrieved_criteria`, `retrieved_context`.

> **Timing note:** this stage's `search_specs` sub-step is currently the slowest fixed cost in the pipeline (~75-80s observed, vs. the desired ~15s) -- this is the BGE-M3 embedding model appearing to reload on every call rather than being cached across the Streamlit process lifetime. Not yet root-caused/fixed as of this writing.

Stage 2 -- `ocr_extract` (Agent 1: OCR Extractor)

src/agents/ocr_extractor.py. Deterministic, no LLM.

- Opens the PDF with PyMuPDF, processes all pages in parallel via `ThreadPoolExecutor(max_workers=8)`.
- Per page: runs `pytesseract.image_to_data` to get word-level bounding boxes, groups words into table rows by y-coordinate (rows below `y > 380` are treated as the hardness-reading table), detects component type (`Pipe`, `Weld`, `HAZ1`, `HAZ2`, `Plate`, `Half Coupling`, `Thermowell`, `End Cap`, `Tee`, etc.) and joint name from each row's left-hand text, and parses numeric hardness readings with their pixel coordinates (needed later for drawing annotation boxes).
- Also extracts page-level `drawing_no` / `spool_no` (regex over the full page text) and a `has_astm_prefix` flag (whether an ASME/ASTM material spec prefix like "SA-335" accompanies a grade like "P91").
- Output: `list[PageOCRResult]` (see §5 State Schema).

Stage 3 -- `resolve_materials` (Agent 2: Material Resolver)

src/agents/material_resolver_agent.py. **Deterministic-first, LLM-on-exception.**

For every row on every page, a two-pass rule-based algorithm assigns a material grade (P91 / P22 / P11 / CS):

- **Pass 1 (grounded):** explicit text match (`\bP22\b`, `\bP11\b`, `\bP91\b|F91|WP91\b`, `\bGr\.?C|CS\b`); a `Plate` row whose readings all fall inside 130-225 HBW is grounded as P22; a row whose readings are *all* above 225 HBW is grounded as P91 (P22/P11/CS's ceiling is 225 -- physically impossible for those grades).
- **Pass 2 (inherit):** rows without their own explicit signal inherit from the *nearest* explicitly-graded row anywhere on the page (not a fixed "row above/below" rule -- real reports usually state the grade once per joint group, often on the middle Weld row), capped at `MAX_INHERIT_DISTANCE = 3` rows so it can't leap across an unrelated dissimilar-metal zone. When two anchors of different materials are equidistant, the tie is broken by which material's range the row's own readings actually fit (not array order). A newly-grounded row becomes an anchor for further rows in the same zone, letting inheritance chain past the 3-row cap within one contiguous run. If the inherited grade conflicts with the row's own readings and P91's wider range doesn't resolve the conflict either, the row is left ungrounded (sent to LLM review) rather than forced.
- **Fallback default:** if nothing grounds a row, it defaults to **P91** (the wider, more permissive range) rather than guessing a narrower grade from drawing-code alone -- policy explicitly chosen to avoid false-positive violations.

Ungrounded/ambiguous rows (mostly OCR-noise outliers) are batched into a single schema-constrained Ollama `/api/chat` call, capped at `MAX_LLM_CANDIDATES_MATERIAL` (env-configurable; current default **0** -- the deterministic rules above resolve nearly everything, so this budget is spent only if explicitly raised).

Stage 4 -- `audit_compliance` (Agent 3: Compliance Auditor)

src/agents/compliance_auditor_agent.py. Same deterministic-first pattern.

- Limits: **P91** 195-248 HBW (base metal/HAZ) or 195-280 HBW (weld); **P22/P11/CS** 130-225 HBW (per spec 700-006 Table 8).
- Calibration/reference rows (every reading in the row is a suspiciously round number from `SKIP_VALUES = (270, 180, 90, 0)`) are skipped entirely.
- **Note 3** (P91 weld): a single reading up to 315 HBW is acceptable if every other reading on that weld is within 195-280.
- **Note 2** (P91 base/HAZ): a single reading down to 180 HBW is acceptable if every other reading is within 195-248.
- Readings within `BORDERLINE_MARGIN_HBW = 15` of a limit are candidates for LLM review, capped at `MAX_LLM_CANDIDATES_COMPLIANCE` (default **2**), prioritized by closest margin to the limit first. Everything else keeps its rule-based PASS/FAIL verdict.
- Output: `list[ComplianceFinding]` with severity `HOLD` (above max) or `MAJOR` (below min).

Stage 5 -- `build_annotations` (Agent 4: Annotation Builder)

src/agents/annotation_builder.py. Purely deterministic, no LLM.

- Converts each `ComplianceFinding` into a display-ready `Finding` dict: human-readable comment/required-action text (re-heat-treatment + re-test for HOLD/above-max, metallurgical review + re-test for MAJOR/below-min), plus a consolidated pixel bounding box (`box`) computed from the coordinates of the specific failing readings, used later by the PDF annotator to draw the callout box.
- Also emits a document-wide finding if no page has an ASME/ASTM material-spec prefix (e.g. "P91" appearing without "SA-335").
- If there are no findings at all, emits a single informational "all criteria satisfied" finding.

Stage 6 -- `synthesize_report` (Agent 5: Report Synthesizer)

src/agents/report_synthesizer_agent.py.

- **Deterministic scoring:** `score = 100 – HOLD×30 – MAJOR×15 – MINOR×5 – missing_doc_control_params×5`, clamped to [0, 100].
- **Disposition thresholds:** any `HOLD` or score < 50 -> **REJECTED**; any `MAJOR` or score < 70 -> **REVISE AND RESUBMIT**; any `MINOR` or score < 90 -> **APPROVED WITH COMMENTS**; else -> **APPROVED**.
- Verifies document-control parameters (project name, PO number, report number, procedure ref) via `verify_doc_control_params` in src/agents/compliance_reviewer.py.
- **One LLM call** here: a single `/api/generate` request (12s timeout) asks the model for a 3-4 sentence executive summary citing the relevant specs, given the finding counts and a sample of reasoning traces from earlier stages. Falls back to a templated deterministic sentence if the call fails or times out.
- Persists two JSON files to `data/parsed/`: `rag_results_latest.json` (what was retrieved during `search_specs`) and `llm_traces_latest.json` (per-stage reasoning traces, for debugging/audit).

---

4. LLM Usage Summary

Call	Stage	Endpoint	Model	Trigger
Material candidate review	`resolve_materials`	`/api/chat` (JSON schema)	`qwen2.5-coder:7b`	Only if ungrounded candidates exist, capped at `MAX_LLM_CANDIDATES_MATERIAL`
Compliance candidate review	`audit_compliance`	`/api/chat` (JSON schema)	`qwen2.5-coder:7b`	Only for borderline readings, capped at `MAX_LLM_CANDIDATES_COMPLIANC`
Executive summary	`synthesize_report`	`/api/generate`	`qwen2.5-coder:7b`	Always (12s timeout, falls back to a template)
Vision page inspection	*(tool exists, not wired into the active graph)*	`/api/generate` (image)	`qwen2.5vl:7b`	Legacy/single-agent path only

Both candidate-review calls send their **entire batch as one schema-constrained request** (not one call per candidate) -- cost scales with total output tokens, not candidate count. On this CPU-only host, throughput is roughly 2-9 tokens/sec depending on thermal/load state, which is why the candidate budgets exist: they're the main lever for keeping total pipeline time predictable (see `src/config.py` for the current tuning rationale and the `OLLAMA_BATCH_TIMEOUT` / `OLLAMA_BATCH_CONCURRENCY` settings).

5. State Schema

All pipeline state lives in one `ReviewState` TypedDict (src/agents/state.py), threaded through every graph node:

Group	Fields
Input	`submittal_text`, `submittal_filename`, `submittal_filepath`
Guide/discipline	`guide_code`, `guide_name`, `cat_code`, `cat_label`
Metadata	`supplier`, `doc_no`, `revision`, `reviewer`, `gen_params_input`
Classification	`doc_type`, `test_method`, `material_grade`
Requirements	`requirements`, `requirement_sources`
Retrieval	`retrieved_criteria`, `retrieved_context`
Pipeline intermediates	`page_ocr_results`, `material_assignments`, `compliance_findings`
Compliance review	`gen_param_checks`, `findings`, `visual_observations`
Gap analysis *(declared, unused)*	`gaps`, `missing_fields`, `ambiguities`
Final report	`disposition`, `score`, `verdict`, `summary`
Diagnostics	`pipeline_metrics`, `llm_traces`, `llm_reasoning_traces`
Agent comms	`messages` (LangGraph `add_messages` reducer), `current_agent`, `errors`

Supporting dataclasses: `Reading` (value + pixel coords), `TableRow` (component, is_weld, left_text, joint_name, readings), `PageOCRResult` (page_idx, full_text, drawing_no, spool_no, table_rows, report_format), `MaterialAssignment` (page/row -> material + reasoning), `ComplianceFinding` (page/row -> failing readings, limit, severity, reasoning). `Finding`/`GenParamCheck` are the display-ready TypedDicts consumed by the UI.

> `gaps`, `missing_fields`, `ambiguities`, `visual_observations` are declared in the state schema but **not populated** by the current linear graph -- `gap_analyzer.py` exists on disk but isn't wired into `_build_langgraph_pipeline`.

6. RAG / Vector Store

- **Embeddings:** `BAAI/bge-m3` (1024-dim), loaded via `langchain-huggingface` in src/vectorstore/embeddings.py. Falls back to a dimension-matched `FakeEmbeddings` if the real model fails to load, to avoid breaking compatibility with existing Chroma collections.
- **Store:** ChromaDB, persisted at `data/chromadb/`, managed by `VectorStoreManager` (src/vectorstore/store.py). Uses a **parent/child** chunking design -- small child chunks are embedded and searched (`qdr_{guide_code}_children`), and full parent chunks (`qdr_{guide_code}_parents`) can optionally be fetched for fuller LLM context.
- **Ingestion:** src/ingestion/pipeline.py (`IngestionPipeline`) glues `parser.py` -> `chunker.py` -> `embeddings.py` -> `store.py`. The Streamlit "Client Procedures" tab calls `ingest_specification(...)` when the user uploads a new spec PDF.

- **Known architectural gap:** `ingest_specification` stores specs under collection key `f"{guide_code}_specs"`, but `search_specs` (the tool actually used during inspection) queries the plain `guide_code` collection -- so specs uploaded via the UI may not actually be retrieved at inspection time. Worth verifying/fixing if RAG-sourced requirements seem stale.

---

7. FastAPI Backend

src/api/main.py -- `FastAPI(title="QDR·AI -- Inspection & RAG API")`, CORS open, runs on port 8000.

Method	Path	Purpose
GET	`/`	Root info (system name, status, docs link)
GET	`/health`	Health check -- reports active LLM/embedding model names
POST	`/inspect`	Runs `run_inspection_pipeline(...)` directly; body = submittal text + metadata; returns the full `ReviewState` as JSON
GET	`/search`	Ad hoc RAG search (`query`, `guide_code`, `k`) against ChromaDB, with parent-chunk expansion
GET	`/logs`	Tail of the pipeline log file (`lines` query param)

The Streamlit UI does **not** call this API -- it imports and calls `run_inspection_pipeline` directly from `app.py`. The API exists for external/ programmatic access to the same pipeline.

---

8. Streamlit UI (`app.py`)

Four tabs, plus a sidebar for discipline/review-mode/guide selection and a run trigger:

1. ** Supplier Document** -- upload or paste the PDF being inspected. Parsed via `parse_pdf(...)`; stores `submittal_text`/`submittal_bytes`/`submittal_filepath` in session state.
2. ** Review Guide & Standards** -- upload the official Review Guide; runs `extract_checklist_items` to split -checked vs -unchecked criteria into `checked_criteria` (fed to `extract_requirements` as the mandatory checklist).
3. ** Client Procedures** -- reference/spec PDFs for RAG (lists the pre-indexed 700-006, 700-016, 700-021, 700-022, 700-026, DEETH-QCP-EL-003 specs); supports uploading additional specs into ChromaDB.
4. ** Inspection Results** -- the report view: - Score/disposition header card + executive summary - Document-control parameter cards (Project Name, PO Number, Report Number, Procedure Ref) - Visual annotated PDF page preview (page selector, flags pages with findings) - Inline findings log (one color-coded card per finding: comment, requirement, required action, closeout status) - Annotated PDF download (generated on click if not already pre-generated)

Run trigger: the " Inspect Document" button is guarded by src/utils/run_lock.py (a process-wide `threading.Lock`) so a second click -- or a Streamlit browser-reload re-executing the script -- can't start a duplicate concurrent pipeline run;

the button shows "■ Inspection Running..." and is disabled while a run is in progress.

---

9. PDF Annotation

src/ingestion/annotator.py draws the final annotated PDF using PyMuPDF (page rendering) + Pillow (ImageDraw). Two rendering modes per page, chosen by finding density (CROWDED_THRESHOLD = 6):

- **Few findings (≤6):** the original per-finding callout-card + pointer-arrow style -- one small box per finding, connected to its table cell by an arrow.
- **Many findings (>6):** one consolidated box at the top of the page listing every finding as a line item, plus up to 6 highlighted target-cell rectangles labeled #N -- used for visually "messy" pages that would otherwise be covered in overlapping boxes.

---

10. Configuration Reference

All settings load from .env via src/config.py:

Variable	Default	Purpose
OLLAMA_MODEL	qwen2.5-coder:7b	Text LLM for all agent calls
OLLAMA_VISION_MODEL	qwen2.5vl:7b	Vision LLM (legacy path only)
OLLAMA_FORCE_LOCAL	true	Skip the (currently unresponsive) remote GPU host probe
OLLAMA_BATCH_TIMEOUT	300	Per-batch-call timeout (seconds) for candidate review calls
OLLAMA_BATCH_CONCURRENCY	1	Parallel Ollama requests -- kept at 1 since this host is CPU-only (no GPU offload, concurrent calls contend rather than parallelize)
MAX_LLM_CANDIDATES_MATERIAL	0	Cap on material-resolver rows sent to the LLM per document
MAX_LLM_CANDIDATES_COMPLIANCE	2	Cap on compliance-auditor rows sent to the LLM per document
TESSERACT_CMD	C:\Program Files\Tesseract-OCR\tesseract.exe	OCR binary path
EMBEDDING_MODEL	BAAI/bge-m3	Embedding model for RAG
EMBED_TOP_K	15	RAG retrieval top-k
AGENT_MODE	langgraph	"langgraph" = this pipeline; anything else = legacy rule-based
QDR_BACKEND_HOST / QDR_BACKEND_PORT	0.0.0.0 / 8000	FastAPI bind address

`CHROMA_PERSIST_DIR`	`./data/chromadb`	Vector store persistence path

The two `MAX_LLM_CANDIDATES_*` caps are the primary lever for total pipeline runtime -- each stage sends its whole candidate batch as **one** schema-constrained Ollama call, so cost scales with total output tokens (`candidates × ~80-100 tokens + 200`), not call count. Tune these first if runtime needs to change.

---

11. Directory Structure

```
QDR-/
+-- app.py                  Streamlit UI (4 tabs + sidebar)
+-- requirements.txt
+-- .env / .env.example
+-- data/
|   +-- documents/          Uploaded/sample submittal & guide PDFs
|   +-- chromadb/           Persisted ChromaDB vector store
|   +-- parsed/             rag_results_latest.json, llm_traces_latest.json
+-- src/
|   +-- config.py           Central environment configuration
|   +-- agents/
|   |   +-- orchestrator.py           LangGraph graph definition + run_inspection_pipeline
|   |   +-- state.py                  ReviewState + supporting dataclasses
|   |   +-- ocr_extractor.py          Agent 1 -- spatial OCR & table extraction
|   |   +-- material_resolver_agent.py  Agent 2 -- material grade resolution
|   |   +-- compliance_auditor_agent.py Agent 3 -- hardness limit compliance
|   |   +-- annotation_builder.py       Agent 4 -- finding -> display/box conversion
|   |   +-- report_synthesizer_agent.py Agent 5 -- scoring, disposition, summary
|   |   +-- tools.py                  classify_document / extract_requirements / search_specs / etc.
|   |   +-- compliance_reviewer.py      Document-control parameter verification
|   |   +-- (legacy/dormant -- not wired into the active graph)
|   |       classifier.py, requirement_extractor.py, gap_analyzer.py,
|   |       crag_evaluator.py, ne_comment_comparator.py, revision_verifier.py,
|   |       report_generator.py, qa_inspector_agent.py, qa_inspector_agent_legacy.py
|   +-- api/
|   |   +-- main.py           FastAPI app (/inspect, /search, /health, /logs)
|   +-- ingestion/
|   |   +-- parser.py         parse_pdf, extract_checklist_items, OCR helpers
|   |   +-- chunker.py        Document chunking for embedding
|   |   +-- pipeline.py       IngestionPipeline (parse -> chunk -> embed -> store)
|   |   +-- annotator.py      annotate_pdf_with_findings -- draws the final PDF
|   |   +-- ingest_folder.py  Bulk/batch spec ingestion script
|   +-- vectorstore/
|   |   +-- embeddings.py     BGE-M3 embedding model singleton
|   |   +-- store.py          VectorStoreManager (ChromaDB parent/child search)
|   +-- utils/
|       +-- logger.py         pipeline_logger, PIPELINE_LOG_FILE
|       +-- metrics.py        StageTracker (per-stage latency logging)
|       +-- run_lock.py       Process-wide lock preventing concurrent pipeline runs
```

---

12. Known Architectural Notes / Limitations

- `search_specs` **embedding reload:** the BGE-M3 embedding model appears to reload on every `search_specs` call (~75-80s observed) rather than being cached across the Streamlit process lifetime -- not yet root-caused.
- **Spec-collection mismatch:** specs ingested via the "Client Procedures" tab are stored under `f"{guide_code}_specs"`, but `search_specs` queries the plain `guide_code` collection at inspection time -- newly-uploaded specs may not actually be retrieved.
- **Dormant modules:** `classifier.py`, `requirement_extractor.py`, `gap_analyzer.py`, `crag_evaluator.py`, `ne_comment_comparator.py`, `revision_verifier.py`, `report_generator.py`, and the single-agent `qa_inspector_agent.py` all exist on disk but are not imported by `_build_langgraph_pipeline` -- only the legacy rule-based fallback (`qa_inspector_agent_legacy.py`) is reachable outside the main graph, and only on an exception.

- **Material inheritance edge case:** the nearest-explicit-grade inheritance in the material resolver is capped at 3 rows per hop specifically to avoid bleeding a material grade across an unrelated, unlabeled joint group -- very sparse pages with a real gap wider than that will still fall back to the P91 default rather than inherit, a known/accepted limitation rather than a bug.
- **Timing variability:** this host runs Ollama on CPU only (no GPU offload); observed throughput for schema-constrained batch calls has ranged 2-9 tokens/sec depending on thermal/load state, and OCR extraction has shown one-off 8x slowdowns attributed to host resource contention rather than the pipeline code itself.